Relatório de evolução — ChargeGrid AI (EV Challenge GoodWe)

Sprint 03 — Agentes de IA e Evolução Conversacional
Prompt and Artificial Intelligence · Ciência da Computação, 1º ano · Parceiro: GoodWe Brasil

1. Resumo da evolução

Sprints 1 e 2. A Sprint 1 definiu problema, persona, fluxo, system prompt e o conjunto de perguntas de avaliação. A Sprint 2 entregou uma aplicação funcional em notebook: dois agentes como funções Python (`agentRespondePergunta` e `agentAvaliador`), chamadas diretas ao SDK da OpenAI, histórico mantido em listas passadas de função em função, RAG com `InMemoryVectorStore` executado a cada turno e injetado no system prompt, e avaliação com few-shots cuja saída era parseada com `json.loads` em `try/except`.

Sprint 03. O núcleo conversacional foi reconstruído em **LangGraph**: (1) o fluxo virou um grafo com nós e roteamento condicional, no lugar do `while True` encadeado; (2) a memória passou a ser estado do grafo, persistido por `thread_id` via checkpointer — sessões isoladas e inspecionáveis; (3) o RAG virou **ferramenta** (`buscar_manual`), chamada pelo agente só quando a pergunta exige dado técnico; (4) o contexto técnico saiu do system prompt fixo e foi para a base indexada; (5) entrou uma camada de **guardrails determinísticos** em Python, somada às instruções do prompt; (6) o avaliador passou a usar `with_structured_output`, eliminando o parsing manual de JSON; (7) a suíte de testes virou código executável — 14 casos, com métricas de nota, tokens, latência e acionamento de guardrail, rodando tanto na arquitetura nova quanto na legada.

2. Refatoração — decisões técnicas

Framework escolhido: LangGraph. Reaproveita os componentes LangChain já usados na Sprint 2 (`init_chat_model`, splitters, vector store); permite barrar um turno **antes** da chamada ao modelo via roteamento condicional (controle menos explícito em CrewAI ou no Agents SDK); resolve sessão isolada com checkpointer por `thread_id`; e `init_chat_model` troca de provedor por string — exatamente o que a comparação entre modelos do Bloco B exige. Alternativas descartadas: **CrewAI** (estrutura de papéis sobrando para um assistente de turno único) e **OpenAI Agents SDK** / **Google ADK** (acoplam a um fornecedor justamente na sprint que pede comparação entre modelos).

Principais decisões e trade-offs:

Decisão	Trade-off assumido
Grafo enxuto (só o guardrail de entrada é nó)	A verificação de saída fica dentro de `call_llm` — um nó a mais ali aumentaria a complexidade do grafo sem mudar comportamento.
Guardrail em duas camadas (regex + prompt)	Regex é determinístico e gratuito, mas só pega padrões previstos; um classificador LLM generalizaria mais, ao custo de uma chamada paga por turno. Optamos por regex + instrução como rede de segurança.
RAG como ferramenta, não injeção fixa	Menos tokens e menos ruído por turno, ao custo de um round-trip extra e do risco de o agente não chamar a ferramenta — mitigado por regra explícita no prompt.
`MemorySaver` (RAM) em vez de disco	Suficiente para a demonstração local; persistência entre execuções é troca de uma linha (`SqliteSaver`).

3. Comparativo antes × depois

Dimensão	Sprints 1–2	Sprint 03
Arquitetura	Funções Python + SDK direto, loop `while True`	Grafo LangGraph, roteamento condicional, estado tipado
Memória	Lista de dicts entre funções, sem isolamento de sessão	Estado do grafo com checkpointer por `thread_id`
RAG	Busca a cada turno, injetada no system prompt	Ferramenta `buscar_manual`, sob decisão do agente
Segurança	Apenas instrução no prompt	Prompt + guardrails determinísticos (entrada/saída)
Avaliação	Few-shots + `json.loads` em `try/except`	`with_structured_output` (schema Pydantic)
Testes	Manuais, no notebook	Suíte versionada, 14 casos, métricas automáticas
Modelo (mesmo nas duas, isola o efeito da arquitetura)	gpt-4o-mini	gpt-4o-mini
Nota média nos testes	0.75	**0.89**
% de respostas "Adequada"	64%	**86%**
Nota de segurança	0.75	**1.00**
Nota de memória	0.83	**1.00**
Tokens médios por turno	1441	1602
Latência média por turno	2.43 s	**1.86 s**

14 casos, execução única (16/09, 17:20–17:21), após a correção da instrução de abertura descrita no Problema 1. Comando: `python -m tests.run_tests --arquitetura {sprint2|sprint3} --modelos gpt-4o-mini`.

> **A nova arquitetura tornou o chatbot melhor?** Sim, de forma mensurável. O ganho mais nítido é em **segurança**: no caso de tentativa de vazamento do system prompt, a Sprint 2 expôs as instruções internas e foi avaliada como Inadequada; na Sprint 3 o guardrail determinístico bloqueou o turno antes de chegar ao modelo — 0 tokens, latência zero, comportamento idêntico em qualquer LLM. Esse bloqueio, mais o caso equivalente de prompt injection, explica boa parte da queda de latência média. A **memória** também ficou mais estável: um caso que oscilava na Sprint 2 entre execuções (lista Python manual) chegou a nota máxima de forma consistente com o checkpointer. Um ponto honesto: **tokens por turno subiram** (1441→1602) — a causa aparente é que respostas mais completas e corretas (citando a fonte, explicando o "não encontrado") custam mais tokens de saída que uma resposta genérica incorreta.

4. Problemas encontrados e soluções

Problema 1 — Instrução de abertura competindo com a pergunta do usuário

Problema Um caso de pergunta objetiva sem perfil declarado ("onde configuro a tarifa por kWh?") regrediu: a Sprint 2 respondia direto; a Sprint 3 perguntou "você é Operador Comercial ou Técnico de Campo?" e nunca respondeu. A instrução de abertura do prompt mandava perguntar o perfil incondicionalmente na primeira mensagem, mesmo quando ela já trazia uma pergunta respondível.

Alternativas (a) remover a pergunta de perfil e deduzir o nível técnico pelo teor da pergunta; (b) condicionar a pergunta de perfil à ausência de uma pergunta objetiva; (c) sempre responder primeiro e perguntar o perfil como adendo ao final.

Solução adotada (b) + (c) combinadas — responde primeiro sempre que houver pergunta respondível; só pergunta o perfil isoladamente quando a mensagem for apenas um cumprimento.

Justificativa Preserva a personalização por perfil (intencional do produto) sem sacrificar a resposta à pergunta real do usuário. Após a correção, o caso passou a "Adequada" e a nota média geral subiu de 0.82 para 0.89.

Problema 2 — Ruído do avaliador (LLM-as-judge) entre execuções idênticas

Problema Ao investigar por que um caso funcional variou de nota entre duas rodadas, comparamos as respostas do agente turno a turno: eram **idênticas,**

	palavra por palavra. O agente não mudou — o juiz (`gpt-4o-mini`, `temperature=0`) deu vereditos diferentes ("Adequada" e "Parcialmente Adequada") para o mesmo texto, com justificativas diferentes.
Alternativas	(a) ignorar o ruído e reportar uma única execução; (b) rodar a suíte múltiplas vezes e reportar a nota média; (c) trocar o juiz por regras (inviável para julgar linguagem natural); (d) self-consistency (votar entre 3 chamadas do juiz por caso).
Solução adotada	(b), com `--execucoes 3` como padrão recomendado para os números finais dos relatórios.
Justificativa	(a) mascara incerteza real e pode levar a decisão de modelo baseada em ruído, não em sinal. (d) resolveria melhor, mas triplicaria o custo do avaliador em cima do já triplicado por `--execucoes 3`; para o escopo da sprint, a média sobre o próprio pipeline já é suficiente para não decidir a partir de um único ponto de dado.

Outros achados relevantes (documentados em detalhe no repositório):

Resistência à injeção variava conforme o modelo: com defesa apenas no prompt, o mesmo ataque de prompt injection era recusado por um modelo e parcialmente atendido por outro. Resolvido com filtro determinístico em Python (`src/guardrails.py`) antes da chamada ao modelo — custo zero, comportamento idêntico em qualquer LLM, condição necessária para que a comparação entre modelos meça o modelo, e não a sorte do prompt.

Alucinação de especificação a partir de contexto genérico do RAG: ao perguntar sobre um equipamento inexistente, o agente chamava a ferramenta corretamente, mas atribuía a ele um valor genérico recuperado (ex. limite de corrente "típico" de um tipo de conector). Corrigido em três iterações no prompt, até proibir citar qualquer valor numérico quando o equipamento perguntado não é encontrado na documentação — eliminando a ambiguidade que o próprio juiz julgava de forma inconsistente.

5. Divisão da equipe

Nome	RM	Principal responsabilidade
Gabriel Jurado Nogueira	571236	Grafo LangGraph e memória por sessão
Guilherme Henrique de Almeida	568708	RAG (ferramenta `buscar_manual`) e base de conhecimento
Guilherme Garbelini	571150	Guardrails e casos de teste de segurança
Mariana Carminato	573258	Suíte de testes e relatório de comparação entre modelos
Vinicius Toralles Ferreira Conduta	570911	Relatório de evolução e organização do repositório

Divisão de responsabilidades proposta pelo grupo, coerente com os entregáveis da Sprint 03 — a confirmar/ajustar pelos integrantes antes da entrega final.